N-Body Simulation
Geavanceerde computerarchitectuur: project

(B-KUL-JLI1QS)

Elias **Neels**, Wout Batselé

May 20, 2026

Year: 2026

Class: MELEC

Instructor: Thomas Feys

Contents

1 Inleiding **2**

2 Achtergrond en Theorie **2**
2.1 Brute-Force 2
2.2 Barnes-Hut Algoritme ($O(N \log N)$) 3
2.3 Leapfrog Integratie 3

3 GPU Architectuur en Optimalisatieconcepten **4**
3.1 Spatial Locality 4
3.2 Memory Coalescing 4
3.3 Thread Divergence 4

4 Architectuur en Implementatiedetails **5**
4.1 Stap 1: Definiëren van de Ruimte (bounding_box.cu) 5
4.2 Stap 2: Sorteren (morton_sort.cu) 5
4.3 Stap 3: Bouwen van de Octree (build_tree_and_com.cu) 5
4.4 Stap 4: Force Calculation (force_calculation.cu) 5
4.5 Stap 5: Integratie (integration.cu) 5
4.6 Graphics Interop en Rendering (graphics_interop.cu) 6

5 Resultaten en Evaluatie **6**

6 Conclusie **6**

A Architectuur Overzicht **7**

1 Inleiding

Een N-body simulatie is een numerieke methode om de dynamische evolutie van een systeem te berekenen dat bestaat uit N deeltjes die continu met elkaar interageren [1]. Deze simulaties zijn essentieel voor diverse wetenschappelijke velden. In de astrofysica worden ze gebruikt om de vorming van sterrenstelsels en de interactie van zwarte gaten te modelleren, terwijl ze op een kleinere schaal worden toegepast voor moleculaire dynamica en eiwitvouwing.
De meest fundamentele en exacte methode voor zo'n simulatie is de *all-pairs* of *brute-force* benadering. Hierbij wordt de kracht tussen elk mogelijk paar van deeltjes berekend. Voor een systeem met N deeltjes leidt dit tot $O(N^2)$ interacties per tijdstap [1]. Hoewel deze methode geschikt is voor kleine systemen of sterk geoptimaliseerde GPU-omgevingen met gedeeld geheugen (Shared Memory Tiling), wordt de berekeningstijd onoverkomelijk lang voor macroscopische systemen (honderdduizenden of miljoenen deeltjes).

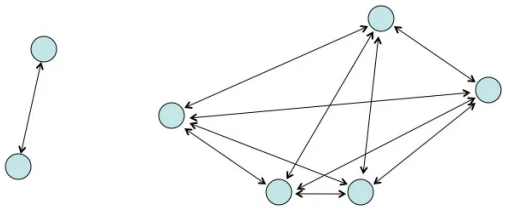


Figure 1: N-boy problem

Om de complexiteit op te schalen naar realistische astronomische proporties in real-time, implementeren we in dit project het veel efficiëntere *Barnes-Hut algoritme*. Deze benadering reduceert de computationele complexiteit van $O(N^2)$ naar $O(N \log N)$ door verre clusters van deeltjes als één enkel 'mega-deeltje' of zwaartepunt (Center of Mass) te behandelen [4]. Het hoofddoel van dit project is het volledig parallel implementeren van deze complexe datastructuur op de parallelle architectuur van een NVIDIA GPU met behulp van CUDA, gekoppeld aan een OpenGL visualisatie.

2 Achtergrond en Theorie

2.1 Brute-Force

De interactie tussen de deeltjes in ons systeem wordt geregeerd door de zwaartekrachtspotentiaal van Newton. Voor een systeem van N deeltjes, elk met een positie $\mathbf{x}_i$ en massa m_i, wordt de krachtvector $\mathbf{f}_{ij}$ die deeltje j op deeltje i uitoefent beschreven door:

$$\mathbf{f}_{ij} = G\frac{m_i m_j}{\|\mathbf{r}_{ij}\|^2} \cdot \frac{\mathbf{r}_{ij}}{\|\mathbf{r}_{ij}\|} \tag{1}$$

Hierbij is $\mathbf{r}_{ij} = \mathbf{x}_j - \mathbf{x}_i$ de vector tussen de twee deeltjes, en G is de gravitatieconstante [1]. De totale versnelling $\mathbf{a}_i$ op deeltje i ontstaat door de interacties met alle andere deeltjes te sommeren, gedeeld door de eigen massa ($\mathbf{a}_i = \mathbf{F}_i/m_i$).
Om numerieke instabiliteit te voorkomen wanneer deeltjes willekeurig dicht bij elkaar komen (wat tot een oneindig grote kracht en een onrealistische afstoting zou leiden), wordt in de theorie

een *softening factor* $\epsilon^2 > 0$ geïntroduceerd. De vergelijking voor de versnelling wordt dan:

$$\mathbf{a}_i \approx G \cdot \sum_{1 \leq j \leq N} \frac{m_j \mathbf{r}_{ij}}{(\|\mathbf{r}_{ij}\|^2 + \epsilon^2)^{3/2}} \tag{2}$$

2.2 Barnes-Hut Algoritme ($O(N \log N)$)

In de plaats van deze sommatie voor élk deeltje N keer uit te voeren, introduceert het Barnes-Hut algoritme een benaderingsmethode op basis van een *Octree* datastructuur (een 3D uitbreiding van een quadtree).
De theorie achter dit algoritme verloopt in verschillende logische stappen [4]:

a. **Octree:** De volledige 3D-ruimte van de simulatie (textitbounding box) wordt recursief opgedeeld in 8 gelijke sub-kubussen (textitoctanten). Dit gaat door totdat elke kubus ofwel leeg is, ofwel precies één deeltje bevat.

b. **Center of Mass:** Elke interne knoop (textitnode) in deze boom berekent zijn totale massa en het gewogen ruimtelijke zwaartepunt van alle deeltjes die zich in de onderliggende takken bevinden.

c. **Tree Traversal:** Voor elk deeltje lopen we door de boom, beginnend bij de root. Bij elke node berekenen we de verhouding s/d, waarbij s de fysieke grootte van de huidige node is, en d de afstand van het deeltje tot het zwaartepunt van de node.

d. **Multipole Acceptance Criterion:** Als de verhouding $s/d < \theta$ (waarbij θ een drempelwaarde is, typisch 0.5), is de cluster ver genoeg verwijderd om beschouwd te worden als één groot, massief deeltje. We stoppen dan het afdalen in die specifieke tak en berekenen de kracht via het zwaartepunt. Is de cluster te dichtbij, dan openen we de node en beoordelen we de 8 onderliggende nodes afzonderlijk.

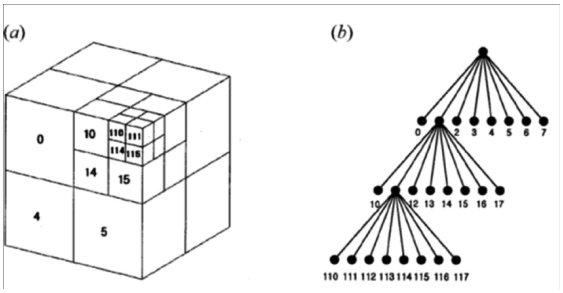


Figure 2: Voorstelling octree

Door deze methodiek hoeft een deeltje aan de rand van de bounding box de posities van de miljoenen individuele deeltjes in het centrum niet te kennen. Het berekent de aantrekkingskracht simpelweg ten opzichte van het zwaartepunt van het centrum.

2.3 Leapfrog Integratie

Om te berekenen hoe de posities in de tijd evolueren aan de hand van de berekende versnellingen, maken we gebruik van de *Leapfrog (kick-drift-kick)* integratiemethode [4]. Deze methode is tijdsymmetrisch (*symplectisch*), wat betekent dat het de totale energie van het systeem veel beter

behoudt tijdens orbitale mechanica dan een standaard *Euler-integrator*. Het algoritme updatet eerst de snelheden over een halve tijdstap (kick), vervolgens de posities over een hele tijdstap (drift), en ten slotte de snelheden over de resterende halve tijdstap.

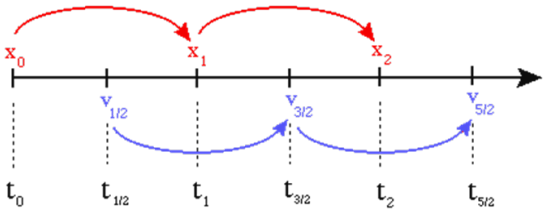


Figure 3: Visualisatie Leapfrog integratiemethode

3 GPU Architectuur en Optimalisatieconcepten

Hoewel het Barnes-Hut algoritme wiskundig veel efficiënter is, is het van nature een *pointer-chasing* algoritme met veel vertakkingen (bomen doorlopen). Dit terwijl de traditionele sterktes van de GPU: opeenvolgende, voorspelbare datastructuren zonder vertakkingen zijn. Om dit gat te overbruggen, passen we optimalisaties toe.

3.1 Spatial Locality

Het grootste pijnpunt van boomstructuren op een GPU is *cache miss-rates*. Als deeltjes willekeurig in het globale geheugen staan, moet een thread willekeurig in het trage VRAM springen, wat bandbreedte verspilt.
Dit lossen we op via *Morton Codes (Z-Order curve)* [4]. Een Morton code mapt een 3D-coördinaat naar een 1D-integer door de bits van de x, y en z coördinaten te "interleaven" (af te wisselen). Door vervolgens onze array van deeltjes te sorteren op basis van deze 1D Morton-waarden, garanderen we dat deeltjes die in de 3D-ruimte dicht bij elkaar liggen, óók in het fysieke geheugen naast elkaar liggen. Wanneer een CUDA warp (32 threads) dus lokale deeltjes inleest, zullen deze in hetzelfde cacheblok zitten.

3.2 Memory Coalescing

Om *coalesced memory access* te verkrijgen, waarbij de hardware de geheugenverzoeken van alle threads in een warp (32 threads) kan bundelen in zo min mogelijk transacties, is data-uitlijning (*alignment*) cruciaal [3]. We vermijden klassieke C++ structs en maken gebruik van het ingebouwde `float4` datatype op de GPU. We bundelen de positie (x, y, z) en de massa in de vier elementen van de `float4`. Deze neemt exact 16 bytes in beslag, waardoor geheugentoegang optimaal via de brede bus van de GPU verloopt en we de latency bij het globale geheugen minimaliseren [3].

3.3 Thread Divergence

Op de GPU draaien 32 threads in een warp perfect synchroon via het SIMT (Single Instruction, Multiple Thread) principe [2]. Als binnen een warp sommige threads een knoop openen (omdat de cluster dichtbij is) terwijl andere threads stoppen (omdat de verhouding $s/d < \theta$ voldoet),

treedt er *Thread Divergence* op [2]. Hardwarematig leidt dit tot serialisatie van de operaties, wat nadelig voor de snelheid is [2].
Omdat we onze deeltjes echter hebben gesorteerd met de Morton curve, zullen opeenvolgende threads in een warp ook deeltjes verwerken die ruimtelijk dicht bij elkaar liggen. Hierdoor hebben zij grofweg hetzelfde perspectief op de boom. Ze zullen in de meeste gevallen over dezelfde knopen beslissen om deze te openen of als zwaartepunt te evalueren. Dit is een subtiele maar bijzonder effectieve manier om thread divergence te minimaliseren tijdens het doorlopen van de boom.

4 Architectuur en Implementatiedetails

De opbouw van onze applicatie is georganiseerd in een *pipeline*, die per frame wordt doorlopen. De CPU coördineert de opeenvolgende CUDA kernels en stuurt uiteindelijk de weergave via OpenGL aan [4].

4.1 Stap 1: Definiëren van de Ruimte (bounding_box.cu)

Voordat we de datastructuur kunnen opbouwen, moeten we weten hoe groot de ruimte is. We gebruiken hiervoor reductie-kernels om in parallel uitersten (`min` en `max` coördinaten) van alle deeltjes op te zoeken [4]. Om *race conditions* tijdens de globale reductie tegen te gaan, maken we tevens gebruik van 'atomicMin' en 'atomicMax' in het globale geheugen.

4.2 Stap 2: Sorteren (morton_sort.cu)

Zodra de bounding box bekend is, transformeren we de coördinaat van elk deeltje naar een 3D Morton code. Vervolgens maken we gebruik van een parallelle Radix Sort implementatie om de arrays met posities en snelheden te sorteren in $O(N \log N)$ tijdscomplexiteit [4].

4.3 Stap 3: Bouwen van de Octree (build_tree_and_com.cu)

Met de gesorteerde data kan de octree worden opgebouwd. Omdat de deeltjes al ruimtelijk gesorteerd zijn op basis van Morton codes, kunnen aaneengesloten blokken van deeltjes eenvoudigweg worden toegewezen aan specifieke vertakkingen van de boom. Zodra de boom is opgebouwd, implementeren we een *bottom-up pass* die, via atomaire operaties, het *Center of Mass* van elke node in de boom accumuleert.

4.4 Stap 4: Force Calculation (force_calculation.cu)

Dit is de kern van het programma. Elk deeltje start een thread die bij de root van de octree begint [4]. We navigeren recursief door de boom (op de GPU geïmplementeerd met een lokale stack-array in *local memory*). Voor elke node evalueren we de drempelwaarde θ. De geaccumuleerde versnelling wordt tijdelijk bijgehouden in thread-lokale variabelen en uiteindelijk weggeschreven naar een globaal array.

4.5 Stap 5: Integratie (integration.cu)

Na de krachtberekening voeren we de eerder beschreven Leapfrog integratie uit [4]. Deze kernel leest de data in een past vervolgens de posities en snelheden aan, dit gebeurt coalesced en onafhankelijk van elkaar (geen thread divergence of memory hazards).

4.6 Graphics Interop en Rendering (graphics_interop.cu)

Wanneer de posities zijn geüpdatet in de GPU, moeten we deze weergeven. Een manier is om de resultaten terug naar de CPU te sturen, om deze vervolgens weer terug te sturen naar OpenGL om ze te tekenen. De betere manier is om gebruik te maken van *CUDA-OpenGL Interoperability* [4]. Op deze manier kan men de CPU overslaan in de renderloop. Door complicaties met WSL konden we dit echter niet implementeren.

5 Resultaten en Evaluatie

De overstap van een all-pairs ($O(N^2)$) methode naar het Barnes-Hut ($O(N \log N)$) algoritme biedt een immense sprong in performantie, met name bij grotere systemen.
Performantiemetrieën en Schaalbaarheid: Waar een brute-force methode bij 65.536 deeltjes meer dan 4 miljard interacties per frame moet berekenen, reduceert Barnes-Hut dit (afhankelijk van θ) tot grofweg $N\times \sim 500$ interacties (32 miljoen interacties per frame). `scenarios/` directory [4]).

Table 1: Prestatie vergelijking van de validation, benchmark, and showcase scenarios (28 één-second samples elk) (Lagere FPS omwille van data collection).

Scenario	Avg FPS	Med FPS	Frame (ms)	BuildTree (ms)	Force (ms)	PackVBO (ms)
Gravity Sanity Check (Stable Orbits)	15.00	15.00	70.49	18.38	13.59	0.76
FPS Benchmark	10.86	10.00	98.67	4.72	15.32	3.92
Stellar Pinwheel Showcase	8.79	10.00	132.90	14.83	11.20	1.94

Korte bespreking: De waargenomen prestatieverschillen worden voornamelijk bepaald door het aantal gesimuleerde deeltjes. De *Gravity Sanity Check* bevat slechts 301 deeltjes (1 centrale massa + 3 ringen van 100), wat de hoogste gemiddelde FPS verklaart. De *FPS Benchmark* bevat 150000 deeltjes en vertoont daarom duidelijk lagere FPS door de grotere rekenlast voor krachtberekening en rendering/packing. De *Stellar Pinwheel Showcase* bevat ongeveer 12806 deeltjes (5 sterrenstelsels van telkens 2501, plus centrale en halo-deeltjes), en ligt qua deeltjesaantal dus tussen beide scenario's in; toch is deze trager dan verwacht omdat er met twee substeps per frame wordt gewerkt en de meer complexe multi-galaxy verdeling extra rekenkracht per frame kost. Het aantal deeltjes is dus de dominante factor, terwijl het aantal substeps en de ruimtelijke verdeling secundaire effecten geven.

6 Conclusie

Samenvatting: In dit project hebben we een snelle N-Body simulatie ontwikkeld. Door gebruik te maken van het Barnes-Hut algoritme in combinatie parallellisme op een GPU en CUDA specifieke optimalisaties (Morton z-order sorting en memory coalescing met 'float4'), slagen we erin om de vertakte boomstructuur op SIMT-hardware met succes aan te pakken.

A Architectuur Overzicht

Om een overzicht te bewaren over de verschillende scenario's (Milky Way, botsende clusters, ...) zijn deze gescheiden van de CUDA code in de `include/scenarios/` map.

References

[1] L. Nyland, M. Harris, en J. Prins, "Chapter 31. Fast N-Body Simulation with CUDA," in *GPU Gems 3*, NVIDIA Corporation, 2007. (Hoewel gebruikt als inspiratie voor zwaartekracht formules, hebben we hier het geavanceerdere Barnes-Hut algoritme voor de GPU geïmplementeerd).

[2] E. Neels en W. Batselé, "Session 3 - Optimization techniques. Geavanceerde computerarchitectuur: labosessie," KU Leuven Gent, Maart 2026.

[3] E. Neels en W. Batselé, "Session 4 - Coalesced memory access and device (GPU) memory types. Geavanceerde computerarchitectuur: labosessie," KU Leuven Gent, Maart 2026.

[4] "N-Body Simulation: Codebase Architecture Overview," Architectuur beschrijving van het huidige project repository, 2026.